

Caracterização da Complexidade Estrutural em Projetos Spring Boot

Engenharia de Software - PUC Minas

Aluno: Guilherme Augusto

Gabriel Ferreira

Contextualização

O Spring Boot é amplamente utilizado em projetos Java, mas há evidências limitadas sobre sua complexidade estrutural em repositórios públicos no github.

- O impacto da arquitetura: Como o uso massivo da Injeção de Dependências (DI), um pilar do framework, afeta os valores tradicionais de acoplamento (CBO e RFC).
- A validade das métricas: Se as métricas clássicas (como CBO) ainda são preditores fortes de qualidade e defeitos neste novo cenário arquitetural, especialmente quando comparadas a variantes mais detalhadas (como acoplamento aferente/eferente - Ca/Ce) ou quando se controla o fator verbosidade (LOC/WMC), que pode mascarar as verdadeiras relações.

Métricas orientadas a objetos (CBO, RFC, DIT, NOC) quantificam atributos como acoplamento, coesão e herança, essenciais para avaliar qualidade de software [1], [2], [4] e [6].

A injeção de dependências (DI), característica central do framework, podem influenciar essas métricas de forma não capturada por ferramentas clássicas, impactando manutenibilidade e incidência de defeitos [8], [9], [10] e [11].

Problema

Falta de caracterização empírica e quantificada da complexidade estrutural e da densidade lógica em projetos Java modernos que utilizam Spring Boot.

Objetivo

Analisar projetos Java open-source desenvolvidos com Spring Boot com o propósito de caracterizar sua complexidade estrutural do ponto de vista de métricas orientadas a objetos, focando em quantificar o acoplamento e a estrutura de herança, e em investigar a relação entre a densidade lógica e a complexidade agregada das classes.

Objetivos Especificos

1. Caracterizar o Acoplamento Médio (Q1):

Avaliar empiricamente o volume e a distribuição do acoplamento entre classes nos projetos Spring Boot, quantificando o CBO (Coupling Between Objects) e o RFC (Response for a Class) para verificar se o uso de Injeção de Dependências (DI) se traduz em níveis de acoplamento moderados e controlados.

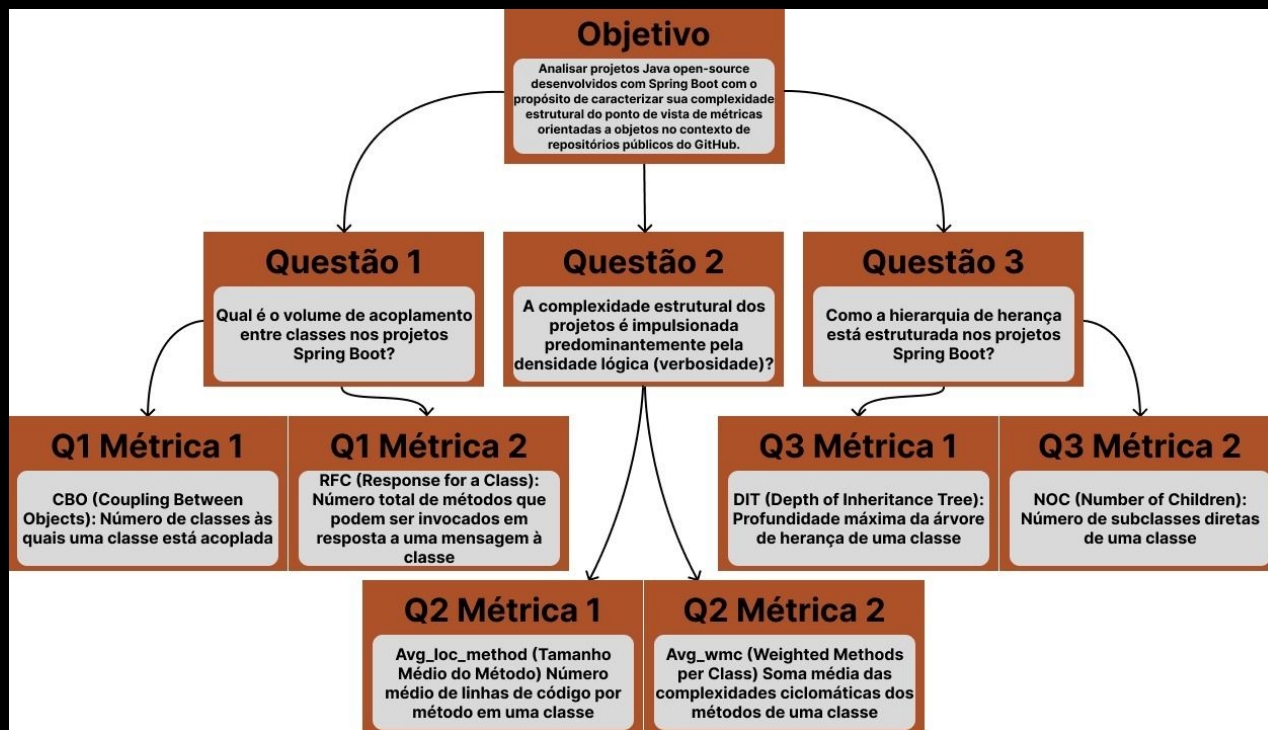
2. Investigar o Impulso da Complexidade (Q2):

Analisar a associação entre a densidade lógica (Tamanho Médio do Método - Avg_loc_method) e a complexidade estrutural agregada das classes (Avg_wmc) para determinar se a verbosidade do código é um fator predominante que impulsiona a complexidade total dos projetos.

3. Quantificar a Estrutura de Herança (Q3):

Descrever a estrutura da hierarquia de herança nos projetos Spring Boot, quantificando o DIT (Depth of Inheritance Tree) e o NOC (Number of Children) para confirmar se o framework induz a um padrão de Composição sobre Herança, resultando em hierarquias rasas.

GQM



Q1. Qual é o volume de acoplamento entre classes nos projetos Spring Boot?

M1.1

CBO (Coupling Between Objects): Número de classes às quais uma classe está acoplada (dependências).

M1.2

RFC (Response for a Class): Número total de métodos que podem ser invocados em resposta a uma mensagem à classe [Singh and Verma 2012, Gupta et al. 2015].

H1.0 (Nula): Não existe um padrão consistente de acoplamento em projetos Spring Boot; CBO e RFC apresentam variação ampla entre projetos, refletindo diferentes abordagens arquiteturais [1.] e [4].

H1.1 (Exploratória): Projetos Spring Boot bem estruturados apresentam níveis de moderados a baixos de CBO e RFC como resultado do padrão arquitetural do framework (camadas Controller-Service-Repository) e do uso extensivo de Dependency Injection, que promove separação de responsabilidades e reduz dependências explícitas entre classes [4].

Q2. A complexidade estrutural dos projetos é impulsionada predominantemente pela densidade lógica (Verbosidade)?

M2.1[⌵]

Avg_loc_method (Tamanho Médio do Método) Número médio de linhas de código por método em uma classe

H2.0 (Nula): Não existe correlação significativa entre o tamanho médio dos métodos (Avg_loc_method) e a complexidade estrutural dos projetos (Avg_wmc); a relação observada decorre de fatores confundidores, como variação no estilo de codificação, padrões arquiteturais adotados e natureza das funcionalidades implementadas [7].

M2.2[⌵]

Avg_wmc (Weighted Methods per Class) Soma média das complexidades ciclomáticas dos métodos de uma classe

H2.1 (Exploratória): Projetos com métodos mais extensos e verbosos (Avg_loc_method elevado) tendem a apresentar maior complexidade estrutural (Avg_wmc), pois a densidade lógica e a granularidade reduzida dos métodos aumentam o peso de complexidade por classe, dificultando legibilidade, manutenção e reuso [7].

Q3. Como a hierarquia de herança está estruturada nos projetos Spring Boot?

M3.1

DIT (Depth of Inheritance Tree): Profundidade máxima da árvore de herança de uma classe

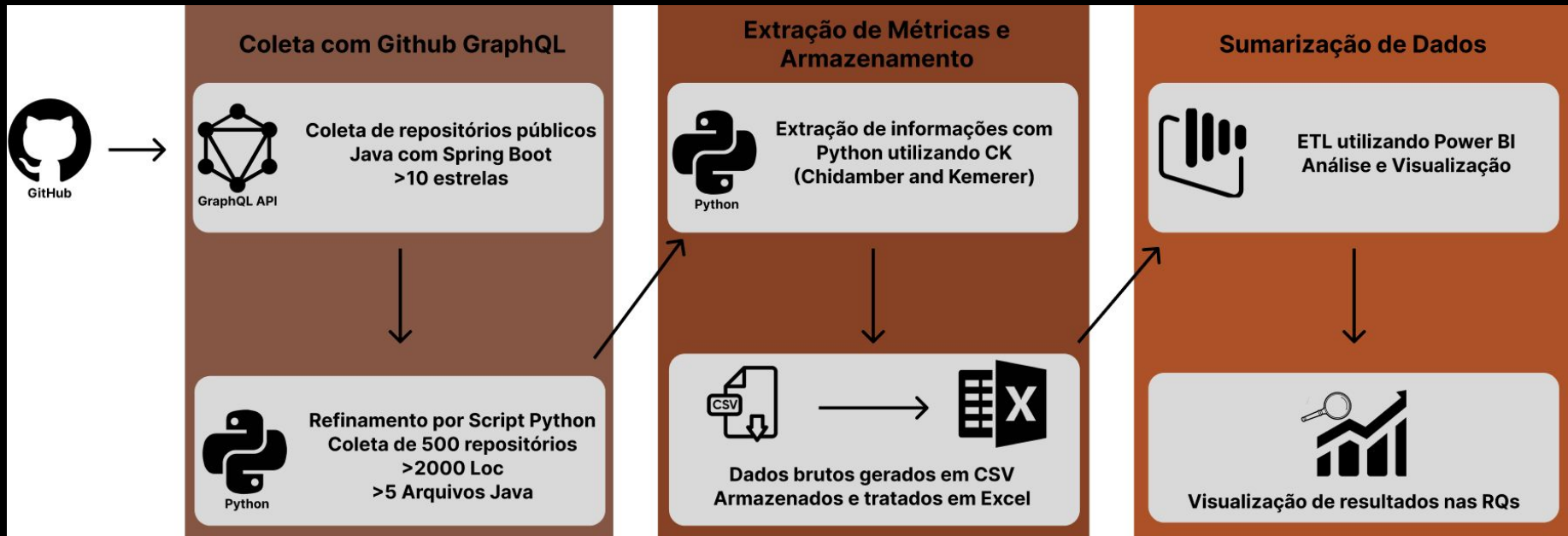
M3.2

NOC (Number of Children): Número de subclasses diretas de uma classe

H3.0 (Nula): Não existe padrão consistente de profundidade de herança em projetos Spring Boot; DIT e NOC apresentam variação considerável [6].

H3.1 (Exploratória): Projetos Spring Boot apresentam hierarquias de herança rasas (DIT reduzido) e NOC baixo devido ao paradigma de composição sobre herança promovido pelo framework, que favorece injeção de dependências e interfaces ao invés de hierarquias complexas de classes [6].

Metodologia



Metodologia

Etapa 1 – Seleção dos Repositórios:

Foram selecionados 500 repositórios públicos de projetos Java desenvolvidos com Spring Boot a partir da GitHub REST API v3, utilizando script Python com paginação automática para busca sistemática. Os critérios de inclusão aplicados foram:

- **Linguagem primária:** Java
- **Framework:** Spring Boot (identificado por presença de dependências spring-boot-starter no arquivo pom.xml ou build.gradle)
- **Popularidade:** mínimo de 10 estrelas (stars), indicador de relevância comunitária
- **Status:** repositórios não-forkados (originais)
- **Tamanho:** mínimo de 2.000 linhas de código (LOC), baseado no critério estabelecido por estudos de débito técnico

Metodologia

Etapa 2 – Extração de Métricas de Código:

Os repositórios selecionados foram clonados localmente via git clone em diretório temporário e submetidos à análise automatizada por meio da ferramenta CK (Chidamber-Kemerer Java Metrics Calculator), versão 0.7.1. A ferramenta CK foi escolhida por ser amplamente validada na literatura de engenharia de software empírica e calcular métricas orientadas a objetos em conformidade com as definições originais de Chidamber e Kemerer.

As métricas extraídas por classe foram:

- **CBO (Coupling Between Objects):** número de classes às quais uma classe está acoplada, considerando dependências de herança, associação e uso de métodos
- **RFC (Response for a Class):** número total de métodos que podem ser invocados em resposta a uma mensagem recebida pela classe, incluindo métodos próprios e chamados
- **DIT (Depth of Inheritance Tree):** profundidade máxima da classe na árvore de herança, medida como o número de níveis até a classe raiz (Object)
- **NOC (Number of Children):** número de subclasses diretas (filhas imediatas) de uma classe
- **WMC (Weighted Methods per Class):** somatório da complexidade ciclomática de todos os métodos da classe
- **LOC:** linhas de código efetivas por classe (excluindo comentários e linhas em branco)

Resultado Bruto: A ferramenta CK gera os Dados brutos em CSV para cada repositório que são posteriormente armazenados e tratados em Excel.

Metodologia

Etapa 3 – Consolidação e Integração dos Dados:

ETL e Análise (Utilizando Power BI):

- **ETL (Extract, Transform, Load):** Os dados brutos consolidados (provenientes das planilhas) são submetidos a um processo de ETL, utilizando a ferramenta Power BI.
- **Power BI:** é empregado para realizar o tratamento final, integração e modelagem dos dados. Isso inclui garantir a consistência dos conjuntos de dados, calcular métricas derivadas (como a média por projeto: Avg_cbo, Avg_wmc) e preparar a estrutura de dados para as análises estatísticas e descritivas necessárias.
- **Análise:** Nesta fase, as análises são realizadas sobre os dados já tratados.

Visualização de Resultados nas RQs:

- São criados painéis e gráficos específicos que ilustram de forma clara e objetiva as métricas.
- Esta etapa garante que os achados empíricos sejam comunicados de maneira eficaz, permitindo a interpretação e a conclusão das hipóteses de pesquisa.

Q1 : Qual é o volume de acoplamento entre classes nos projetos Spring Boot?

M1 : CBO (Coupling Between Objects):
Número de classes às quais uma classe está acoplada (dependências)

1,37

Mínimo

4,81

1° Quartil

5,61

Mediana

5,73

Média

6,51

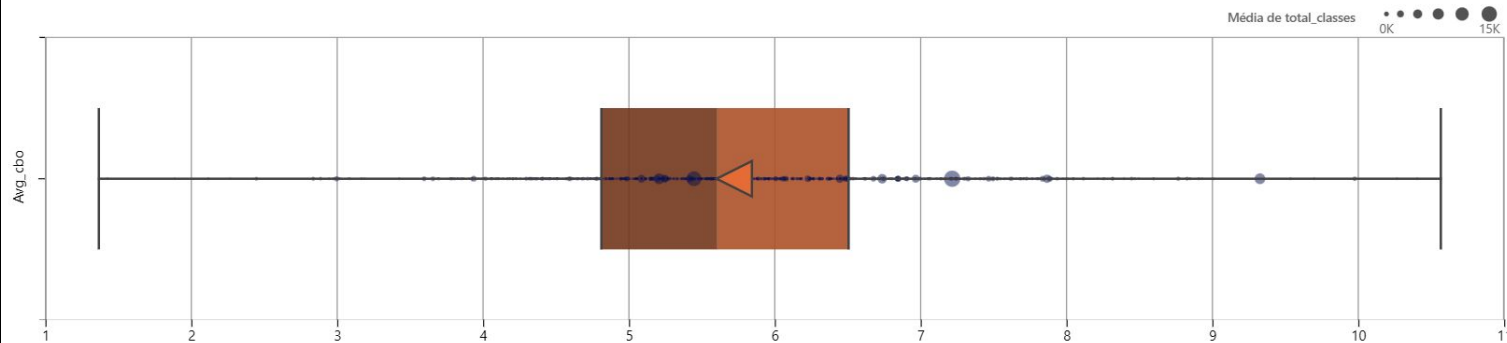
3° Quartil

10,57

Máximo

5,73

Avg_cbo por item



Q1 : Qual é o volume de acoplamento entre classes nos projetos Spring Boot?

M2 : RFC (Response for a Class):

Número total de métodos que podem ser invocados em resposta a uma mensagem à classe

1,05

Mínimo

6,24

1° Quartil

8,19

Mediana

8,97

Média

10,84

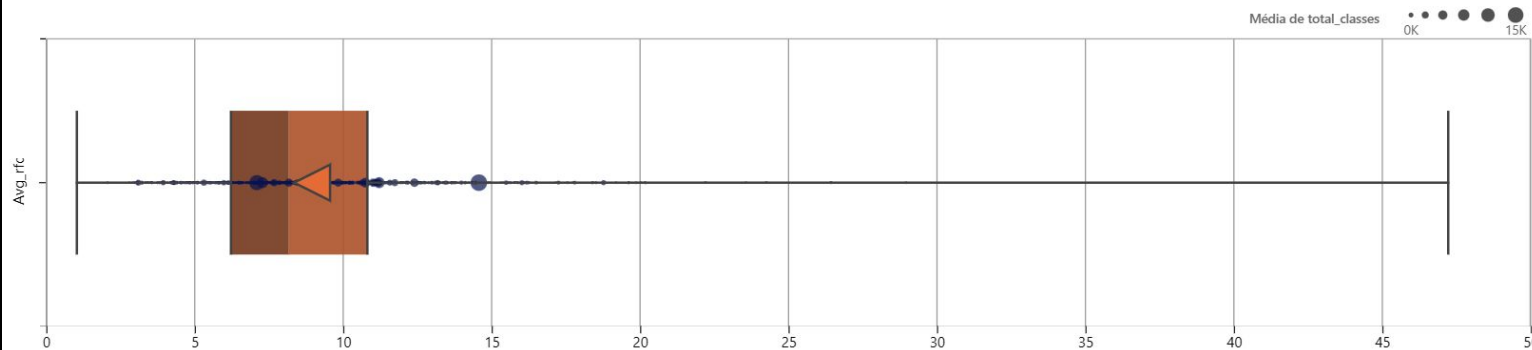
3° Quartil

47,24

Máximo

8,97

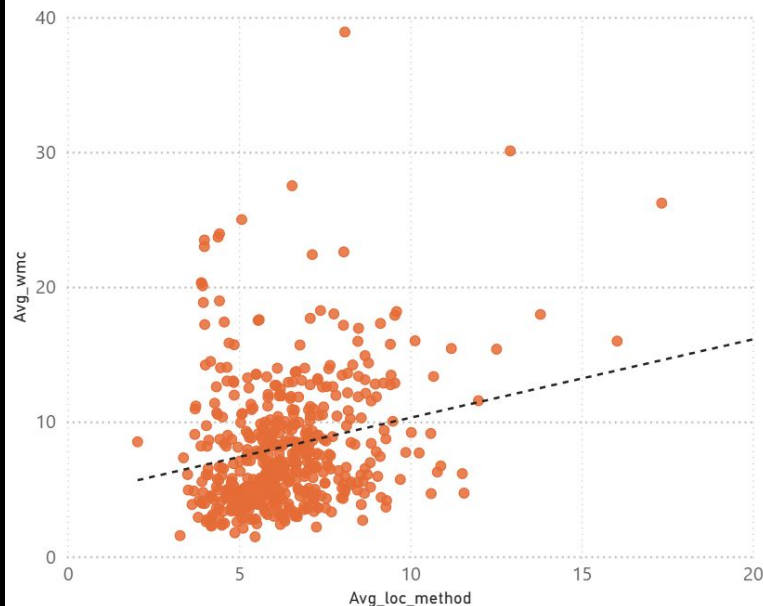
Avg_rfc por item



Q2 : A complexidade estrutural dos projetos é impulsionada predominantemente pela densidade lógica (verbosidade)?

M1: avg_loc_method (Tamanho Médio do Método)

M2: avg_wmc (Weighted Methods per Class)



26,81

Máximo de avg_loc_method

69,90

Máximo de avg_wmc

6,50

Média de avg_loc_method

8,26

Média de avg_wmc

6,14

Mediana de avg_loc_method

6,74

Mediana de avg_wmc

2,04

Mínimo de avg_loc_method

1,47

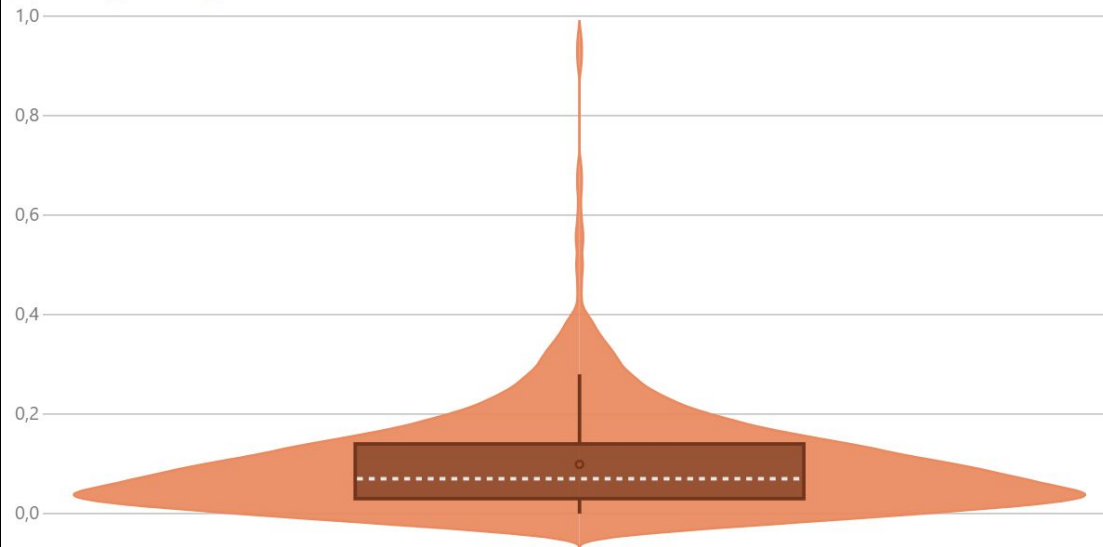
Mínimo de avg_wmc

Q3. Como a hierarquia de herança está estruturada nos projetos Spring Boot?

M1: NOC (Number of Children):
Número de subclasses diretas de uma classe

Média de avg_noc por repository

▲ Avg_noc ■ Median Value ■ Mean Value



0,14

3° Quartil

0,93

Máximo

0,07

Mediana

0,10

Média de avg_noc

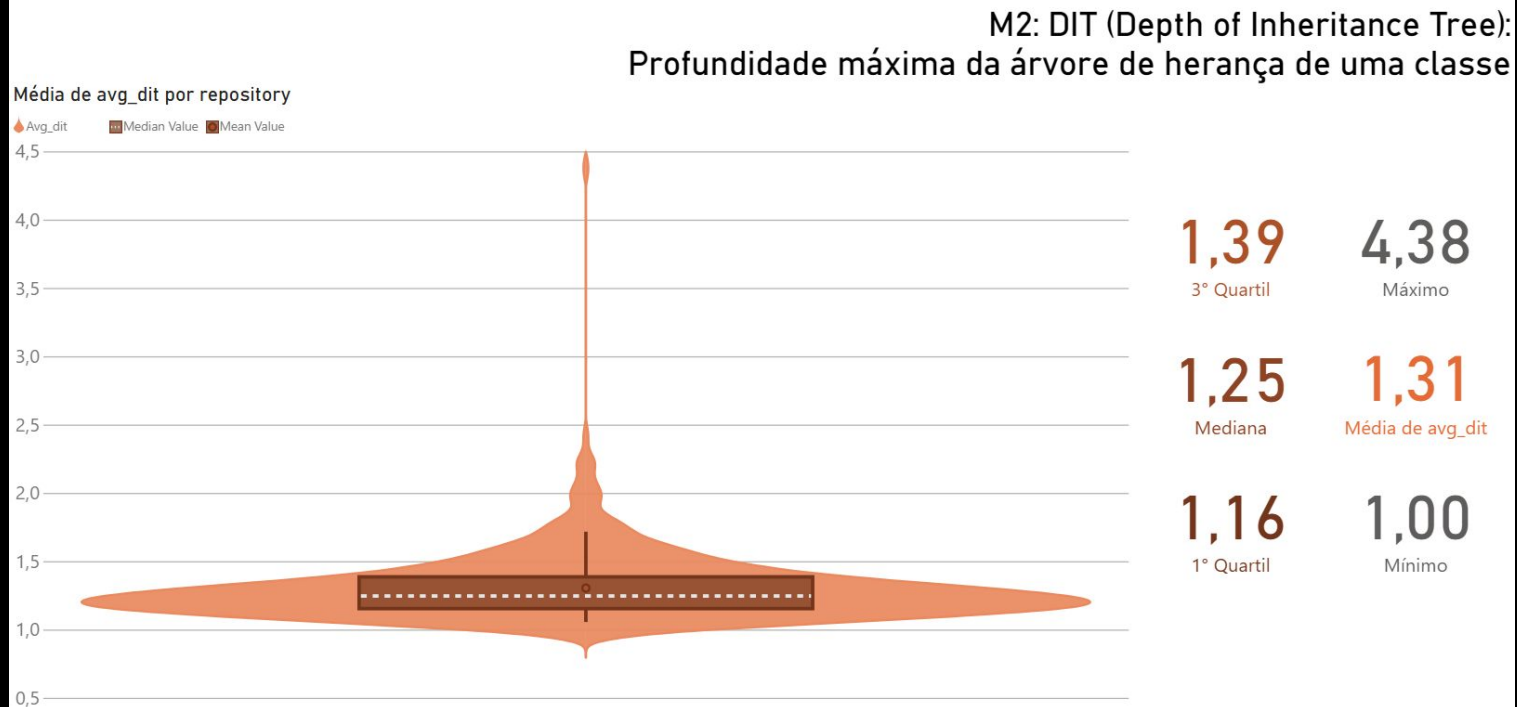
0,03

1° Quartil

0,00

Mínimo

Q3: Como a hierarquia de herança está estruturada nos projetos Spring Boot?



Ameaças à Validade

- Construto: Dependência da ferramenta CK e possíveis limitações em capturar relações mediadas por injeção de dependências.
- Interna: Tamanho, estilo e organização do código podem influenciar as métricas e gerar confundimentos.
- Externa: Amostra filtrada de 500 repositórios do GitHub pode gerar viés de seleção e limitar a generalização.
- Estatística: Uso de métricas agregadas por repositório pode ocultar variações internas e padrões específicos.
- Procedimental: Falhas de clonagem, repositórios indisponíveis e diferenças de ambiente podem afetar a amostra.

Resumo da Discussão

- Acoplamento geralmente moderado, com alguns casos extremos sugerindo excesso de responsabilidades.
- Métodos longos aumentam complexidade (WMC) e destacam a importância de maior decomposição.
- Hierarquias de herança rasas predominam, refletindo o estilo de composição do Spring Boot.
- Forte variação entre repositórios indica inconsistências em boas práticas.
- Métricas ajudam a entender impactos futuros na manutenção e qualidade interna.

Conclusão

Maturidade da Arquitetura (Q1 e Q3):

- O Spring Boot cumpre bem seu papel de organizar o projeto. Os dados evidenciam baixo acoplamento [5] (as classes são independentes e pouco “amarradas” umas às outras — CBO Médio ~5,7).
- Além disso, a hierarquia é rasa (evita estruturas complexas de “pais e filhos”, preferindo simplicidade – DIT ~1,3), confirmando o bom uso da Injeção de Dependência.

O Desafio do Tamanho dos Métodos (Q2):

- Embora a estrutura seja boa, a complexidade interna (o quão difícil a classe é de entender — WMC) aumenta devido à verbosidade (excesso de linhas de código dentro dos métodos — LOC).
- Existe uma relação clara: métodos muito longos geram classes complexas e difíceis de dar manutenção.

Resumo do Estudo:

- O framework organiza a comunicação entre as partes, mas não “arruma” a lógica interna. A qualidade final depende do desenvolvedor, aplicar Clean Code (código limpo) e refatoração para manter as funções pequenas e focadas.

Referências

- [1] Mike Child, Peter Rosner, Steve Counsell, A comparison and evaluation of variants in the coupling between objects metric, Journal of Systems and Software, Volume 151, 2019, Pages 120-132, ISSN 0164-1212, <https://doi.org/10.1016/j.jss.2019.02.020>.
- [2] Filó, T. G. S., Bigonha, M. A. S., & Ferreira, K. A. M. (2024). Evaluating Thresholds for Object-Oriented Software Metrics. Journal of the Brazilian Computer Society, 30(1), 315–346. <https://doi.org/10.5753/jbcs.2024.3373>.
- [3] Olague et al. – "Empirical Validation of Object-Oriented Metrics in Industrial Environment" (2007).
- [4] S. Anwer, A. Adbellatif, M. Alshayeb and M. S. Anjum, "Effect of coupling on software faults: An empirical study," 2017 International Conference on Communication, Computing and Digital Systems (C-CODE), Islamabad, Pakistan, 2017, pp. 211-215, doi: 10.1109/C-CODE.2017.7918930.
- [5] HEMPEL, Gabriel Gaspar. Utilizando instrumentação na extração de métricas de aplicações JAVA em tempo de execução. 31f. 2020. Unicesumar - Universidade Cesumar: Maringá 2020.
- [6] Singh, P. and Verma, S. (2012). Empirical investigation of fault prediction capability of object oriented metrics of open source software. In 2012 Ninth International Conference on Computer Science and Software Engineering (JCSSE), pages 323–327.
- [7] Tahir, A., Bennin, K. E., Xiao, X., and MacDonell, S. G. (2021). Does class size matter? an in-depth assessment of the effect of class size in software defect prediction. Empirical Software Engineering, 26(5):106.
- [8] Sun, P., Kim, D.-K., Ming, H., and Lu, L. (2022). Measuring impact of dependency injection on software maintainability. Computers, 11(9).

Referências

- [9] Rodrigo Laigner, Diogo Mendonça, Alessandro Garcia, Marcos Kalinowski, Cataloging dependency injection anti-patterns in software systems, Journal of Systems and Software, Volume 184, 2022, 111125, ISSN 0164-1212, <https://doi.org/10.1016/j.jss.2021.111125>. (<https://www.sciencedirect.com/science/article/pii/S0164121221002223>).
- [10] N. Santos and A. Rito Silva, "A Complexity Metric for Microservices Architecture Migration," 2020 IEEE International Conference on Software Architecture (ICSA), Salvador, Brazil, 2020, pp. 169-178, doi: 10.1109/ICSA47634.2020.00024.
- [11] Chenxing Zhong, He Zhang, Chao Li, Huang Huang, Daniel Feitosa, On measuring coupling between microservices, Journal of Systems and Software, Volume 200, 2023, 111670, ISSN 0164-1212, <https://doi.org/10.1016/j.jss.2023.111670>. (<https://www.sciencedirect.com/science/article/pii/S0164121223000651>).

Obrigado!